Modelo Conceptual e Relacional - Padaria

Bases de dados (L.EIC012)

Luís Barbosa (up202303872)
Luís Gonçalves (up202305120)
Sofia Sousa (up202303662)

Contextualização

Este relatório pretende definir um modelo conceptual de uma base de dados para uma padaria. A base de dados abarca todo o processo de fabrico e de venda de produtos, pelo que se representam dados relativos à confecção de produtos, à sua distribuição e venda.

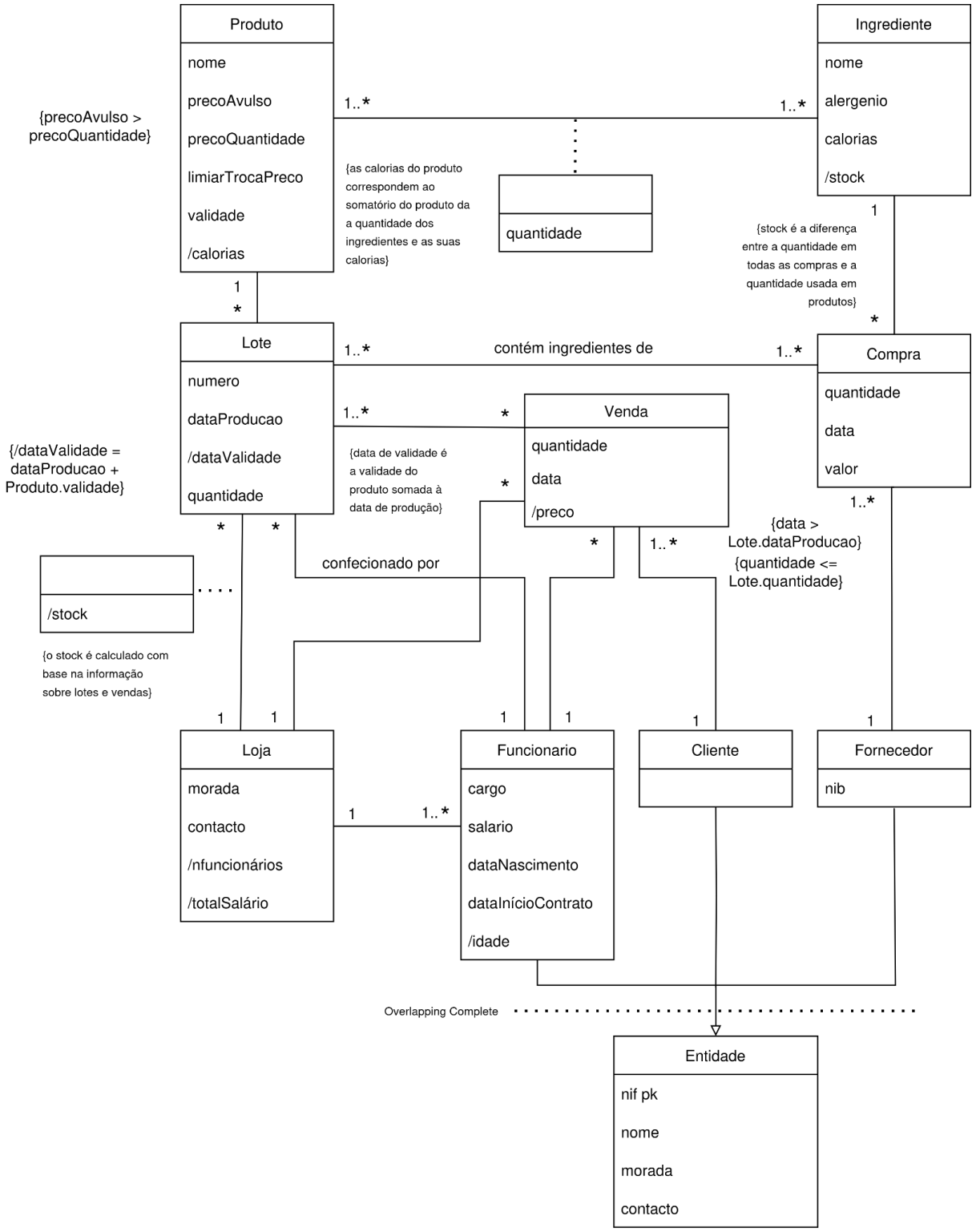
O aspeto mais relevante da base de dados são os diferentes produtos oferecidos. O seu preço depende da quantidade comprada, dado que compras em maior quantidade usufruem de preços reduzidos. Sobre um produto é também necessário que esteja disponível informação sobre o teor calórico e o prazo de validade a partir da data em que é confeccionado.

Tendo em mente garantir a segurança alimentar, a produção é organizada em lotes, permitindo rastrear a origem dos ingredientes utilizados e, no caso de contaminações, quais os produtos afetados.

Quanto aos ingredientes, regista-se se são alergénios e a quantidade de calorias por quilograma, a partir da qual se calcula o teor calórico dos produtos que integra. Os ingredientes são adquiridos a um fornecedor, sobre o qual se guarda informação para pagamentos e contacto.

Finalmente, à padaria estão associadas várias lojas com diferentes moradas e contacto próprio. Pretende-se também compilar estatísticas sobre que funcionários trabalham numa loja e consequente despesa salarial total em cada sucursal. Cada funcionário tem um cargo e salário definidos e é registada a data de início de contrato. Os produtos são vendidos a clientes numa dada loja por um determinado funcionário. É possível documentar clientes, a que se associa um nome, morada, contacto e informação fiscal (NIF).

Modelo Conceptual Inicial



Integração de IA Generativa

Prompt:

Este relatório pretende definir uma base de dados para uma padaria. A base de dados abarca o processo de fabrico e de venda de produtos, pelo que se representam dados relativos à confecção de produtos, à sua distribuição e venda. O aspeto mais relevante da base de dados são os diferentes produtos oferecidos. O seu preço depende da quantidade comprada, dado que compras em maior quantidade usufruem de preços reduzidos. Sobre um produto é também necessário que esteja disponível informação sobre o seu teor calórico e o prazo de validade a partir da data em que são confecionados. Tendo em mente garantir a segurança alimentar, a produção é organizada em lotes, permitindo rastrear a origem dos ingredientes utilizados e, no caso de contaminações, quais os produtos afetados. Quanto aos ingredientes, regista-se se são alergénios e a quantidade de calorias por quilograma, a partir do qual se calcula o teor calórico dos produtos que integra. Os ingredientes são adquiridos a um fornecedor, sobre o qual se guarda informação para pagamentos e contacto. Finalmente, à padaria estão associadas várias lojas com diferentes moradas e contacto próprio. Pretende-se também compilar estatísticas sobre que funcionários trabalham numa loja e consequente despesa salarial total em cada sucursal. Cada funcionário tem um cargo e salário definidos e é importante registar a data de início de contrato.. Os produtos são vendidos a clientes numa dada loja por um determinado funcionário. É possível registar clientes, a que se associa um nome, morada, contacto e informação fiscal (NIF).

Gera as classes necessárias para representar esta base de dados e descreve as relações entre estas.

ChatGPT:

Para representar a base de dados da padaria, vamos definir as classes necessárias e descrever suas relações. Aqui está uma estrutura de classes que abrange os requisitos mencionados:

Classes

1. **Produto**
   - **Atributos:**
     - id: int
     - nome: string
     - preco: decimal
     - teorCalorico: int
     - validade: int (em dias)

- **Métodos:**
  - calcularPreco(quantidade: int): decimal (calcula o preço com base na quantidade)

2. **Ingrediente**
   - **Atributos:**
     - id: int
     - nome: string
     - alergenio: boolean
     - caloriasPorKg: int
   - **Métodos:**
     - calcularCalorias(quantidade: decimal): int (calcula as calorias com base na quantidade utilizada)
3. **Fornecedor**
   - **Atributos:**
     - id: int
     - nome: string
     - contacto: string
     - infoPagamento: string
   - **Métodos:**
     - realizarPagamento(valor: decimal): void
4. **Loja**
   - **Atributos:**
     - id: int
     - nome: string
     - morada: string
     - contacto: string
   - **Métodos:**
     - adicionarFuncionario(funcionario: Funcionario): void
5. **Funcionario**
   - **Atributos:**
     - id: int
     - nome: string
     - cargo: string
     - salario: decimal
     - dataInicio: date
   - **Métodos:**
     - calcularDespesaSalarial(): decimal
6. **Cliente**
   - **Atributos:**
     - id: int
     - nome: string
     - morada: string
     - contacto: string
     - NIF: string

   - **Métodos:**
     - fazerCompra(produto: Produto, quantidade: int): void

7. **Lote**
    - **Atributos:**
        - id: int
        - dataProducao: date
        - quantidadeProduzida: int
    - **Métodos:**
        - rastrearOrigem(): void

Relações entre as Classes

1. **Produto - Ingrediente:**
    - **Relação:** Um produto é composto por um ou mais ingredientes.
    - **Tipo:** Composição (associação de muitos para muitos).
2. **Produto - Lote:**
    - **Relação:** Um produto é associado a um ou mais lotes de produção.
    - **Tipo:** Composição (um para muitos).
3. **Ingrediente - Fornecedor:**
    - **Relação:** Um ingrediente é fornecido por um fornecedor.
    - **Tipo:** Associação (muitos para um).
4. **Loja - Funcionario:**
    - **Relação:** Uma loja pode ter vários funcionários.
    - **Tipo:** Composição (um para muitos).
5. **Loja - Produto:**
    - **Relação:** Uma loja pode vender vários produtos.
    - **Tipo:** Associação (muitos para muitos).
6. **Funcionario - Cliente:**
    - **Relação:** Um funcionário pode realizar vendas para vários clientes.
    - **Tipo:** Associação (um para muitos).
7. **Cliente - Loja:**
    - **Relação:** Um cliente compra produtos em uma loja específica.
    - **Tipo:** Associação (muitos para um).

Resumo

As classes acima refletem a estrutura necessária para modelar a base de dados da padaria, incluindo as relações entre os produtos, ingredientes, fornecedores, lojas, funcionários e clientes. Essa estrutura permite gerenciar eficientemente o processo de fabricação e venda, assegurando a rastreabilidade e segurança alimentar.

Prompt:

Adiciona classes separadas para vendas e compras.

ChatGPT:

Para incorporar as classes de **Venda** e **Compra** na estrutura da base de dados da padaria, podemos definir as classes e suas relações com as outras entidades. Aqui está a atualização:

Classes Adicionais

1. **Venda**
    - **Atributos:**
        - id: int
        - data: date
        - valorTotal: decimal
        - funcionario: Funcionario (associação com a classe Funcionario)
        - cliente: Cliente (associação com a classe Cliente)
    - **Métodos:**
        - calcularValorTotal(): decimal (soma os preços dos produtos vendidos)
2. **Compra**
    - **Atributos:**
        - id: int
        - data: date
        - quantidade: int
        - produto: Produto (associação com a classe Produto)
        - lote: Lote (associação com a classe Lote)
    - **Métodos:**
        - calcularCustoTotal(): decimal (cálculo do custo com base na quantidade e preço do produto)

Atualização nas Relações

1. **Venda - Funcionario:**
    - **Relação:** Uma venda é realizada por um funcionário.
    - **Tipo:** Associação (muitos para um).
2. **Venda - Cliente:**
    - **Relação:** Uma venda é feita para um cliente específico.
    - **Tipo:** Associação (muitos para um).
3. **Venda - Produto:**
    - **Relação:** Uma venda pode incluir vários produtos.
    - **Tipo:** Composição (muitos para muitos). Para representar isso, pode-se criar uma classe intermediária chamada **ItemVenda**.
    - **Classe Intermediária:**
        - **ItemVenda**
            - **Atributos:**
                - produto: Produto
                - quantidade: int
                - precoUnitario: decimal
4. **Compra - Produto:**
    - **Relação:** Uma compra é feita de um produto específico.
    - **Tipo:** Associação (muitos para um).
5. **Compra - Lote:**
    - **Relação:** Uma compra pode estar associada a um lote específico.
    - **Tipo:** Associação (muitos para um).

Resumo Atualizado

As novas classes **Venda** e **Compra** ajudam a gerenciar o processo de vendas e aquisições de produtos na padaria, permitindo um controle mais detalhado das transações. A inclusão da classe **ItemVenda** facilita a gestão das

vendas de múltiplos produtos em uma única transação. A estrutura geral da base de dados agora inclui:

- **Produto**
- **Ingrediente**
- **Fornecedor**
- **Loja**
- **Funcionario**
- **Cliente**
- **Lote**
- **Venda**
- **Compra**
- **ItemVenda** (classe intermediária)

Essa organização proporciona uma visão clara e funcional do sistema de gerenciamento da padaria, garantindo rastreabilidade, controle de estoque e facilitação das vendas.

Prompt:

A classe compra refere-se à compra de um ingrediente a um fornecedor. O que muda no modelo?

ChatGPT:

Se a classe **Compra** refere-se à aquisição de ingredientes de fornecedores (em vez de produtos da padaria), algumas mudanças precisam ser feitas no modelo. A compra agora vai estar associada à aquisição de **ingredientes** e à relação com os **fornecedores**, em vez de produtos e lotes de produção. Além disso, será necessário ajustar a rastreabilidade para identificar quais ingredientes foram comprados de quais fornecedores. Aqui está como o modelo atualizado ficaria:

Atualizações Necessárias

1. Classe Compra

A classe **Compra** agora estará relacionada com a aquisição de ingredientes para a produção da padaria. Isso significa que ela se associa diretamente à classe **Ingrediente** e **Fornecedor**. Veja os novos atributos e métodos:

Compra (atualizada)

- **Atributos**:
  - id: int
  - data: date
  - quantidade: decimal (quantidade do ingrediente adquirido)
  - ingrediente: Ingrediente (associação com a classe Ingrediente)
  - fornecedor: Fornecedor (associação com a classe Fornecedor)
  - precoUnitario: decimal (preço do ingrediente por unidade de medida)
- **Métodos**:
  - calcularCustoTotal(): decimal (calcula o custo total da compra com base na quantidade e no preço unitário)

2. Atualizações nas Relações

Aqui estão as alterações nas relações devido à mudança no conceito de **Compra**:

- **Compra - Ingrediente**:
  - **Relação**: Uma compra refere-se a um ingrediente específico.
  - **Tipo**: Associação (muitos para um). Um ingrediente pode ser comprado várias vezes.
- **Compra - Fornecedor**:
  - **Relação**: Uma compra é feita de um fornecedor específico.
  - **Tipo**: Associação (muitos para um). Um fornecedor pode fornecer muitos ingredientes.

3. Classe Lote

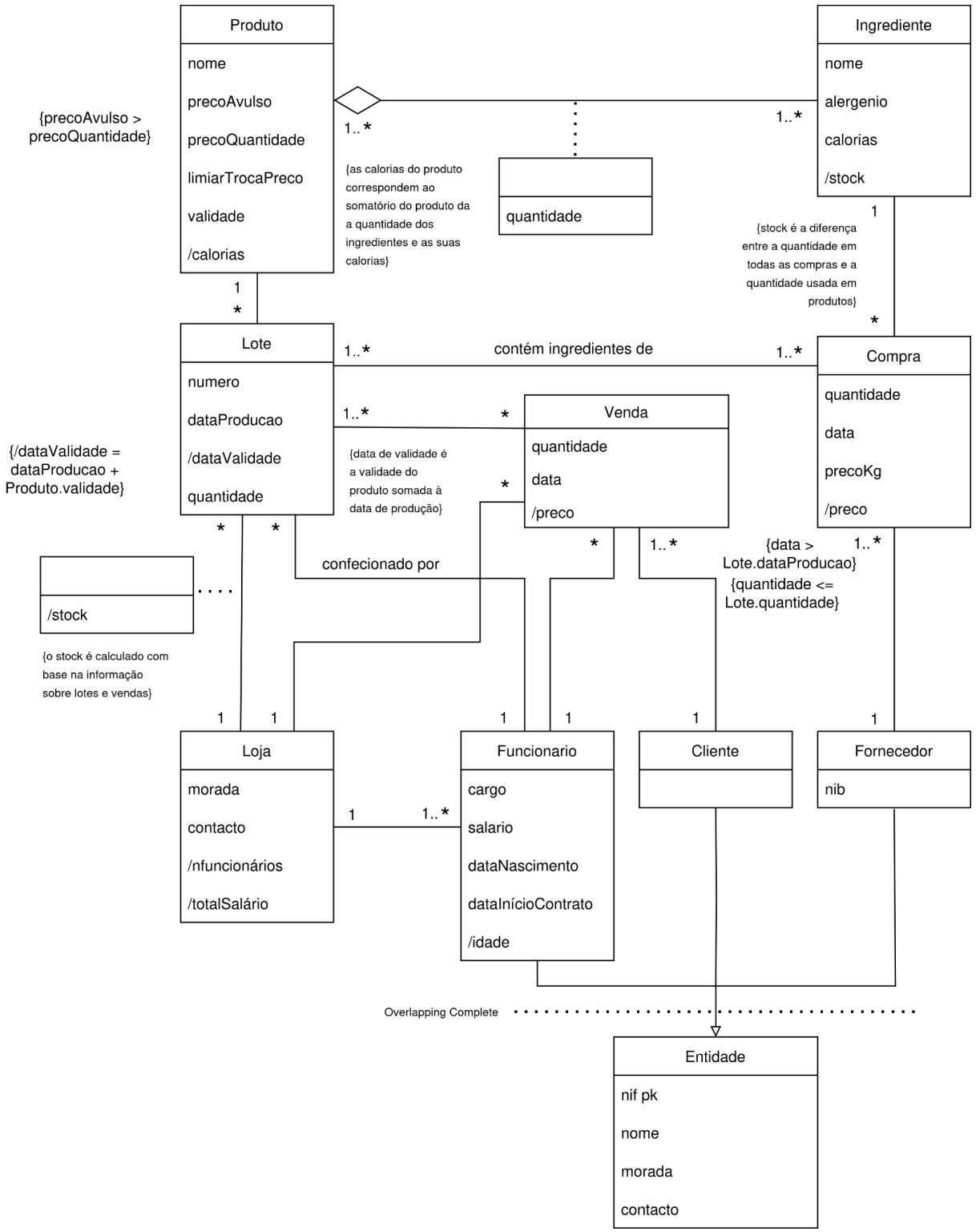
A classe **Lote** também pode ser afetada, pois, em vez de ser relacionada diretamente à **Compra** (que agora se refere a ingredientes), ela deve manter sua relação com a produção dos **Produtos**. Dessa forma, o **Lote** continua associando um conjunto de produtos a uma produção específica, e não a uma compra de ingredientes.

Apreciação do modelo sugerido por IA

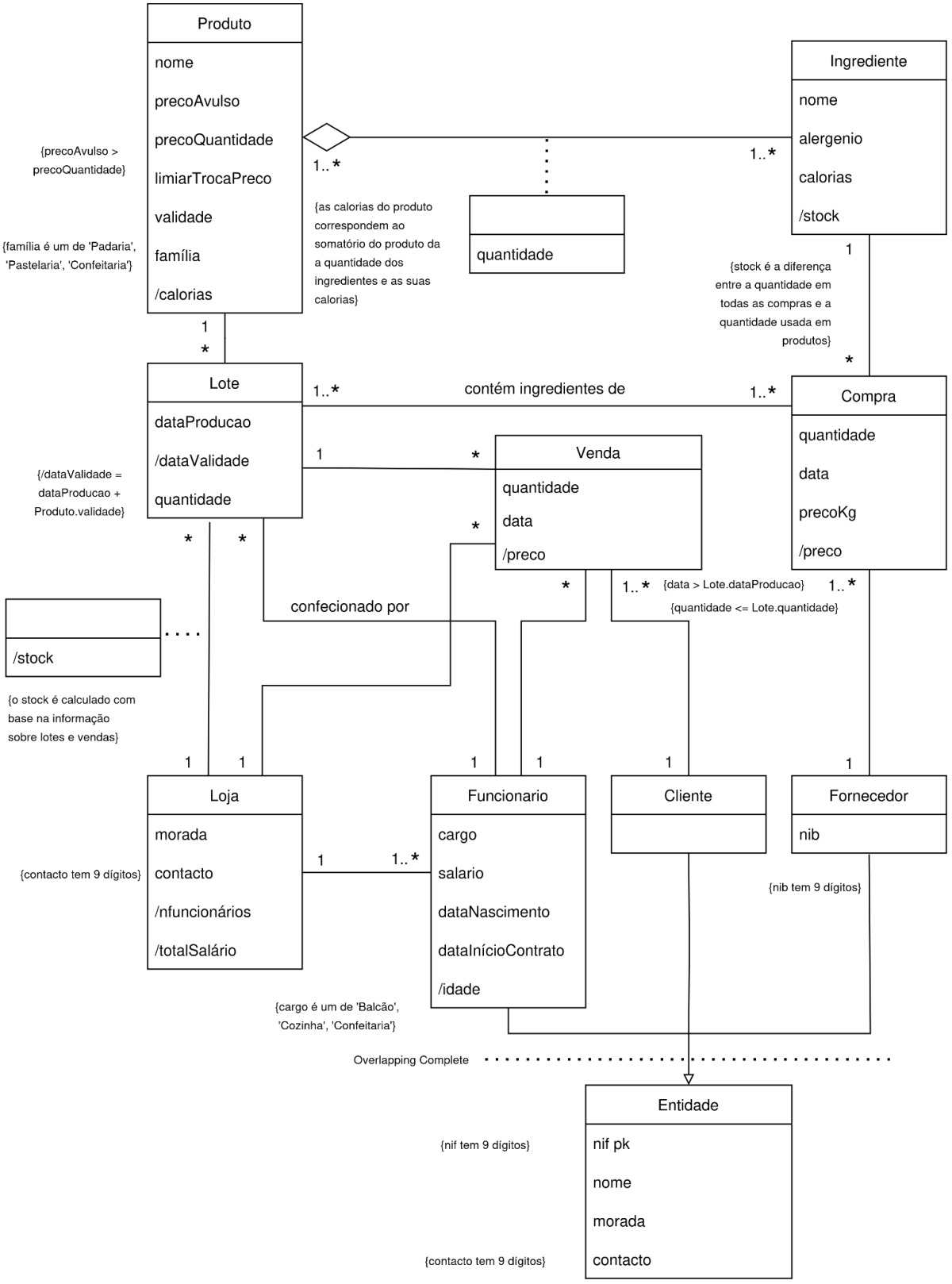
O modelo conceptual sugerido por Inteligência Artificial coincidiu em grande parte com o modelo que concebemos originalmente. Das diferenças realça-se a ausência da generalização de entidades, assim como a omissão de atributos de algumas classes: a distinção entre preço de produtos para venda avulso e em quantidade, os atributos derivados para lojas e lotes e idade para funcionários.

Para além disso, foram sugeridos métodos, que nós não implementámos no âmbito deste projeto, e várias associações de composição entre classes. Implementámos apenas a relação de agregação entre produtos e ingredientes, dado que as restantes não nos pareceram adequadas. No entanto, a relação sugerida não inclui a classe de associação com o atributo quantidade, que mantivemos. Também adicionamos o atributo precoKg à classe Compra, transformando o preço total num atributo derivado.

Modelo Conceptual Final



Modelo Conceptual Refinado



Esquema relacional

Esquema inicial

Entidade (nif, nome, morada, contacto)

Cliente (nif->Entidade.nif)

Fornecedor (nif->Entidade.nif, nib)

Ingrediente (id, nome, alergenio, calorias, stock)

Compra (id, ingrediente->Ingrediente.id, fornecedor->Fornecedor.nif, data, quantidade, precoKg, preco)

Produto (id, nome, precoAvulso, precoQuantidade, limiarTrocaPreco, validade, calorias)

ProdutoIngrediente (produto->Produto.id, ingrediente->Ingrediente.id, quantidade)

Loja (id, morada, contacto, numeroFuncionarios, totalSalario)

Funcionario (nif->Entidade.nif, cargo, salario, dataNascimento, dataInicioContrato, idade, loja->Loja.id)

Lote (id, produto->Produto.id, funcionario->Funcionario.nif, quantidade, dataProducao, dataValidade)

LoteCompra (lote->Lote.numero, compra->Compra.id)

LoteLoja (lote->Lote.id, loja->Loja.id, stock)

Venda (id, lote->Lote.id, loja->Loja.id, funcionario->Funcionario.nif, cliente->Cliente.nif, quantidade, data, preco)

Integração de IA Generativa

Nota: O modelo conceptual refinado, criado no draw.io, foi exportado como um ficheiro `.html` *e fornecido ao ChatGPT. Curiosamente, o entendimento deste formato era mais preciso quando as* prompts *eram feitas em inglês, pelo que é essa a língua usada nesta secção. Para além disso, o ChatGPT mostrou dificuldades em perceber algumas das associações mais complexas no modelo, pelo que as mesmas também foram fornecidas na* prompt *inicial.*

Prompt:

Convert this UML schema of a Bakery's database into a relational model. The format used should be the following (example with a college and its students):

- Student (SID, SName, Grade)
- College (CName, State, Enrollment)
- Applied (SID->Student, CName->College)

Please keep in mind that:

- Funcionario, Cliente and Fornecedor inherit from Entidade.
- The relation between Produto and Ingrediente is a many to many one, with an association class with attribute quantidade.
- The relation between Lote and Stock is of the type many to one with an association class with attribute stock.
- The relation between Lote and Compra is many to many.
- Venda relates to Loja, Lote, Funcionario and Cliente.

ChatGPT:

Based on the provided UML schema and your specifications, here is the conversion into a relational model following the given format:

- Entidade (NIF PK, Nome, Morada, Contacto)
- Funcionario (NIF->Entidade, Cargo, Salario, DataNascimento, DataInicioContrato)
- Cliente (NIF->Entidade)
- Fornecedor (NIF->Entidade, NIB)
- Produto (ProdutoID PK, Nome, PrecoAvulso, PrecoQuantidade, LimiarTrocaPreco, Validade, Calorias)
- Ingrediente (IngredienteID PK, Nome, Alergenio, Calorias)
- Produto_Ingrediente (ProdutoID->Produto, IngredienteID->Ingrediente, Quantidade)
- Lote (LoteID PK, DataProducao, DataValidade, Quantidade)
- Compra (CompraID PK, Data, PrecoKg, Quantidade)
- Compra_Lote (CompraID->Compra, LoteID->Lote)
- Stock (LoteID->Lote, StockQuantidade)
- Venda (VendaID PK, Data, Quantidade, Preco)
- Loja (LojaID PK, Morada, Contacto, numeroFuncionarios, TotalSalario)
- Venda_Details (VendaID->Venda, LojaID->Loja, LoteID->Lote, NIF_Funcionario->Funcionario, NIF_Cliente->Cliente)

Each line represents a table, where PK denotes the primary key, and arrows (->) indicate foreign key relationships. The many-to-many relationships are captured through associative tables with relevant attributes (e.g., Produto_Ingrediente for Produto and Ingrediente).

Prompt:

The UML schema explicitly connects Lote and Loja directly. What relational schema would reflect that?

ChatGPT:

If the UML schema directly connects Lote and Loja, you should create a direct relationship between these entities in the relational model. Here's how you could reflect that connection in the relational schema:

- Add a LojaID foreign key in the Lote table to directly link Lote to Loja.
- Update the Lote table definition to include this relationship.

Now, the Lote table has a direct relationship with Loja through the LojaID foreign key. This allows you to directly query which Loja is associated with each Lote, and hence, to find the Loja related to any given Produto by joining Produto to Lote through Produto_Ingrediente.

Prompt:

How do Funcionários relate to Loja? And why does Venda_Details exist?

ChatGPT:

Funcionários' Relation to Loja: Since Loja has attributes like numeroFuncionarios and TotalSalario, which aggregate information about the Funcionários in each Loja, we can infer that each Funcionario is associated with a specific Loja. However, because this is not explicitly shown in the tables, adding a foreign key in the Funcionario table to Loja would make this relationship clear. Updated Funcionario table:

- Funcionario (NIF->Entidade, Cargo, Salario, DataNascimento, DataInicioContrato, LojaID->Loja)

With this, each Funcionario would directly reference a Loja, clarifying which Loja they work in.

Purpose of Venda_Details: The Venda_Details table exists because Venda involves multiple entities with potentially complex relationships. It associates a Venda with specific details, such as:

- Loja: the Loja where the sale took place.
- Lote: the Lote of products sold in the Venda.
- Funcionario: the employee (Funcionario) responsible for the Venda.
- Cliente: the client (Cliente) purchasing the Produto.

This table acts as an associative or "junction" table that captures all these many-to-many and hierarchical relationships within a Venda. By using Venda_Details, you can:

- Track which Funcionario handled each Venda.
- Identify which Lotes were included in each Venda.
- Determine which Loja and Cliente were involved in each Venda.

In summary, Venda_Details centralizes and organizes these relationships, allowing more precise and complex queries about the specifics of each sale.

Prompt:

Please explain the scheme used when Funcionario, Cliente and Fornecedor inherit from Entidade and why that choice was made.

ChatGPT:

The schema chosen for Funcionario, Cliente, and Fornecedor inheriting from Entidade in the relational model is a common approach called Single Table Inheritance (STI), where entities with shared attributes are represented with a base table (Entidade) while specific entity types (e.g., Funcionario, Cliente, Fornecedor) have separate tables linked by a foreign key to the base table.

Explanation of the Scheme

- Entidade Table: Entidade acts as the base table containing common attributes shared by all entities, such as NIF, Nome, Morada, and Contacto. NIF is used as a unique identifier for each entity in the Entidade table, ensuring that each entity (regardless of type) has a distinct identifier across the system.
- Specialized Tables (Funcionario, Cliente, and Fornecedor): Each specialized entity type has its own table with attributes specific to that type. For example:
- Funcionario table includes fields such as Cargo, Salario, DataNascimento, and DataInicioContrato.
- Fornecedor table includes NIB, a bank identifier, which only suppliers need.

These tables include NIF as a foreign key pointing to Entidade. This foreign key creates a one-to-one relationship with Entidade, allowing each Funcionario, Cliente, and Fornecedor record to reference a single Entidade entry.

Reasons for Choosing This Scheme

- Normalization and Data Integrity
- Clear Representation of Relationships
- Flexibility for Entity Expansion
- Logical Querying

This scheme combines Single Table Inheritance benefits with a relational model's efficiency and maintainability. It is a commonly adopted method when entities share a set of core attributes but also require distinct, type-specific attributes or relationships.

Apreciação e esquema final

As alterações propostas são na sua maioria redundantes, sendo mesmo, por vezes, contraproducentes, como é o exemplo de uma tabela adicional que serve apenas para guardar as referências estrangeiras da relação Venda. Por isso, não foram realizadas alterações ao modelo original.

Análise de Dependências Funcionais e de Formas Normais

Análise inicial

Entidade(nif, nome, morada, contacto)

- nif : nome, morada, contacto

Esta relação está em Boyce-Codd Normal Form (BCNF) já que a dependência funcional tem, no seu lado esquerdo, uma superchave. Por consequência, está também em Third Normal Form (3NF).

Cliente(nif->Entidade.nif)

Esta relação está em BCNF já que não tem dependências funcionais não triviais.

Fornecedor(nif->Entidade.nif, nib)

- nif : nib

Esta relação está em BCNF já que a dependência funcional tem, no seu lado esquerdo, uma superchave.

Ingrediente(id, nome, alergenio, calorias, stock)

- id : nome, alergenio, calorias, stock
- nome : id, alergenio, calorias, stock

Esta relação está em BCNF já que ambas as dependências funcionais têm, no seu lado esquerdo, uma superchave.

Compra(id, ingrediente->Ingrediente.id, fornecedor->Fornecedor.nif, data, quantidade, precoKg, preco)

- id : ingrediente, fornecedor, data, quantidade, precoKg, preco

Esta relação está em BCNF já que a dependência funcional tem, no seu lado esquerdo, uma superchave.

Produto(id, nome, precoAvulso, precoQuantidade, limiarTrocaPreco, familia, validade, calorias)

- id : nome, precoAvulso, precoQuantidade, limiarTrocaPreco, familia, validade, calorias
- nome : id, precoAvulso, precoQuantidade, limiarTrocaPreco, familia, validade, calorias

Esta relação está em BCNF já que ambas as dependências funcionais têm, no seu lado esquerdo, uma superchave.

ProdutoIngrediente (produto->Produto.id, ingrediente->Ingrediente.id, quantidade)

- produto, ingrediente : quantidade

Esta relação está em BCNF já que a dependência funcional tem, no seu lado esquerdo, uma superchave.

Loja (id, morada, contacto, numFuncionarios, totalSalario)

- id : morada, contacto, numFuncionarios, totalSalario
- morada : id, contacto, numFuncionarios, totalSalario
- contacto : id, morada, numFuncionarios, totalSalario

Esta relação está em BCNF já que todas as dependências funcionais têm, no seu lado esquerdo, uma superchave.

Funcionario (nif->Entidade.nif, cargo, salario, dataNascimento, dataInicioContrato, idade, loja->Loja.id)

- nif : cargo, salario, dataNascimento, dataInicioContrato, idade, loja

Esta relação está em BCNF já que a dependência funcional tem, no seu lado esquerdo, uma superchave.

Lote (id, produto->Produto.id, funcionario->Funcionario.nif, quantidade, dataProducao, dataValidade)

- id : produto, funcionario, quantidade, dataProducao, dataValidade

Esta relação está em BCNF já que a dependência funcional tem, no seu lado esquerdo, uma superchave.

LoteCompra (lote->Lote.numero, compra->Compra.id)

Esta relação está em BCNF já que não tem dependências funcionais não triviais.

LoteLoja (lote->Lote.id, loja->Loja.id, stock)

- lote : loja, stock

Esta relação está em BCNF já que a dependência funcional tem, no seu lado esquerdo, uma superchave.

Venda (id, lote->Lote.id, loja->Loja.id, funcionario->Funcionario.nif, cliente->Cliente.nif, quantidade, data, preco)

- id : lote, loja, funcionario, cliente, quantidade, data
- funcionario : loja

Esta dependência funcional causa uma violação da BCNF já que funcionario não é uma superkey da relação. A 3NF é também violada porque funcionario não é uma superchave da relação e loja não pertence a uma chave.

- lote : loja

Esta dependência funcional causa uma violação da BCNF já que lote não é uma superkey da relação. A 3NF é também violada porque lote não é uma superchave da relação e loja não pertence a uma chave.

Normalização da classe Loja

Dependências funcionais cujo lado direito tem um atributo:

- id : lote
- id : loja
- id : funcionario
- id : cliente
- id : quantidade
- id : data
- funcionario : loja
- lote : loja

Verificação de dependências funcionais redundantes:

- $\{id\}^+$ = {id, loja, funcionario, cliente, quantidade, data} - Não contém lote logo a dependência funcional é essencial.
- $\{id\}^+$ = {id, lote, funcionario, cliente, quantidade, data, loja} - Contém loja logo a dependência funcional não é essencial.
- $\{id\}^+$ = {id, lote, loja, cliente, quantidade, data} - Não contém funcionario logo a dependência funcional é essencial.
- $\{id\}^+$ = {id, lote, loja, funcionario, quantidade, data} - Não contém cliente logo a dependência funcional é essencial.
- $\{id\}^+$ = {id, lote, loja, funcionario, cliente, data} - Não contém quantidade logo a dependência funcional é essencial.
- $\{id\}^+$ = {id, lote, loja, funcionario, cliente, quantidade} - Não contém data logo a dependência funcional é essencial.

- $\{funcionario\}^+ = \{funcionario\}$ - Não contém loja logo a dependência funcional é essencial.
- $\{lote\}^+ = \{lote\}$ - Não contém loja logo a dependência funcional é essencial.

Assim, as dependências funcionais essenciais são:

- id : lote
- id : funcionario
- id : cliente
- id : quantidade
- id : data
- funcionario : loja
- lote : loja

Dado que não há dependências com mais do que um atributo do lado esquerdo, esta é uma base mínima. Devemos então criar as relações:

- **A** (id, lote, funcionario, cliente, quantidade, data)
- **B** (funcionario, loja) (Funcionario com menos atributos)
- **C** (lote, loja) (Já existe)

A relação **B** já existe no modelo relacional de forma ligeiramente diferente, é a relação Funcionario com apenas 2 atributos, funcionario e loja. A relação **C**, por sua vez, é a relação LoteLoja. Como id é chave de **A**, então não é necessária mais nenhuma relação. Assim, devemos modificar a relação Venda excluindo o seu atributo loja para que esta fique em 3NF.

Verificação:

Venda(id, lote->Lote.id, funcionario->Funcionario.nif, cliente->Cliente.nif, quantidade, data, preco)

- id : lote
- id : funcionario
- id : cliente
- id : quantidade
- id : data

Como id é superchave da relação, esta está em BCNF (e 3NF). Assim, o modelo relacional após a análise inicial de dependências funcionais e formas normais é o seguinte:

- Entidade(nif, nome, morada, contacto) - BCNF
- Cliente(nif->Entidade.nif) - BCNF
- Fornecedor(nif->Entidade.nif, nib) - BCNF
- Ingrediente(id, nome, alergenio, calorias, stock) - BCNF
- Compra(id, ingrediente->Ingrediente.id, fornecedor->Fornecedor.nif, data, quantidade, precoKg, preco) - BCNF
- Produto(id, nome, precoAvulso, precoQuantidade, limiarTrocaPreco, validade, calorias) - BCNF
- ProdutoIngrediente(produto->Produto.id, ingrediente->Ingrediente.id, quantidade) - BCNF
- Loja(id, morada, contacto, numeroFuncionarios, totalSalario) - BCNF
- Funcionario(nif->Entidade.nif, cargo, salario, dataNascimento, dataInicioContrato, idade, loja->Loja.id) - BCNF
- Lote(id, produto->Produto.id, funcionario->Funcionario.nif, quantidade, dataProducao,
- dataValidade) - BCNF
- LoteCompra(lote->Lote.numero, compra->Compra.id) - BCNF
- LoteLoja(lote->Lote.id, loja->Loja.id, stock) - BCNF
- Venda(id, lote->Lote.id, funcionario->Funcionario.nif, cliente->Cliente.nif, quantidade, data, preco) - BCNF

Integração de IA generativa

Prompt:

Este é o modelo relacional de uma base de dados em que foram encontradas as seguintes dependências funcionais:

[Modelo obtido no passo anterior, incluindo as dependências funcionais]

Verifica se todas as dependências funcionais são válidas, remove as que não são e adiciona eventuais novas dependências. Após isso, verifica se as relações estão em BCNF ou 3NF.

ChatGPT:

Para verificar se todas as dependências funcionais (DF) fornecidas são válidas e se as relações estão em BCNF ou 3NF, vamos seguir uma abordagem estruturada. Vamos analisar as dependências, removê-las se não forem válidas, adicionar dependências novas e depois determinar a normalização das tabelas.

1. Análise das dependências funcionais

- 1.1 Entidade(nif, nome, morada, contacto)
  - nif : nome, morada, contacto
    Esta dependência é válida, pois o NIF (Número de Identificação Fiscal) identifica unicamente uma entidade, e, portanto, a partir do NIF, podemos determinar o nome, morada e contacto.

  Conclusão para a entidade: A dependência funcional está válida.

- 1.2 Cliente(nif->Entidade.nif)
  - Sem dependências funcionais não triviais. Não há dependências não triviais para a relação Cliente, pois ela apenas faz referência à tabela Entidade através de uma chave estrangeira. Está em conformidade com o modelo relacional.
- 1.3 Fornecedor(nif->Entidade.nif, nib)
  - nif : nib
    Esta dependência funcional é válida, pois o NIF do fornecedor pode identificar unicamente o seu NIB.

  Conclusão para Fornecedor: A dependência funcional está válida.

- 1.4 Ingrediente(id, nome, alergenio, calorias, stock)
  - id : nome, alergenio, calorias, stock
    A dependência é válida, pois o id do ingrediente é a chave primária e determina unicamente os outros atributos.
  - nome : id, alergenio, calorias, stock
    A dependência nome : id, alergenio, calorias, stock também é válida se considerarmos que o nome do ingrediente pode ser único (assumindo que o nome é único, por exemplo, Sal ou Azeite). No entanto, se o nome não for único, esta dependência não seria válida, pois o nome não determinaria unicamente os outros atributos.

Conclusão para Ingrediente: A dependência funcional nome : id, alergenio, calorias, stock pode ser inválida caso o nome não seja único para cada ingrediente.

- 1.5 Compra(id, ingrediente->Ingrediente.id, fornecedor->Fornecedor.nif, data, quantidade, precoKg, preco)
  - id : ingrediente, fornecedor, data, quantidade, precoKg, preco
    Esta dependência é válida, pois o ID da compra determina unicamente os atributos relacionados à compra.

  Conclusão para Compra: A dependência funcional está válida.

- 1.6 Produto(id, nome, precoAvulso, precoQuantidade, limiarTrocaPreco, familia, validade, calorias)
  - id : nome, precoAvulso, precoQuantidade, limiarTrocaPreco, familia, validade, calorias
    Esta dependência é válida, pois o ID do produto determina unicamente os outros atributos.
  - nome : id, precoAvulso, precoQuantidade, limiarTrocaPreco, familia, validade, calorias
    A dependência nome : id, precoAvulso, precoQuantidade, limiarTrocaPreco, familia, validade, calorias pode ser válida se o nome do produto for único.

  Conclusão para Produto: A dependência funcional nome : id, precoAvulso, precoQuantidade, limiarTrocaPreco, familia, validade, calorias deve ser revista dependendo da unicidade do nome do produto.

- 1.7 ProdutoIngrediente(produto->Produto.id, ingrediente->Ingrediente.id, quantidade)
  - produto, ingrediente : quantidade
    A dependência é válida, pois a combinação de produto e ingrediente determina a quantidade do ingrediente utilizado em determinado produto.

  Conclusão para ProdutoIngrediente: A dependência funcional está válida.

- 1.8 Loja(id, morada, contacto, numFuncionarios, totalSalario)
  - id : morada, contacto, numFuncionarios, totalSalario
    A dependência é válida, pois o ID da loja determina unicamente os outros atributos.
  - morada : id, contacto, numFuncionarios, totalSalario
    A dependência morada : id, contacto, numFuncionarios, totalSalario pode ser inválida, pois não é garantido que a morada seja única.

  Conclusão para Loja: A dependência funcional morada : id, contacto, numFuncionarios, totalSalario não é válida, pois a morada pode não ser única.

- 1.9 Funcionario(nif->Entidade.nif, cargo, salario, dataNascimento, dataInicioContrato, idade, loja->Loja.id)

    - nif : cargo, salario, dataNascimento, dataInicioContrato, idade, loja
      A dependência é válida, pois o NIF é único para cada funcionário e determina os outros atributos.

  Conclusão para Funcionario: A dependência funcional está válida.

- 1.10 Lote(id, produto->Produto.id, funcionario->Funcionario.nif, quantidade, dataProducao, dataValidade)
    - id : produto, funcionario, quantidade, dataProducao, dataValidade
      A dependência é válida, pois o ID do lote determina unicamente os outros atributos.

  Conclusão para Lote: A dependência funcional está válida.

- 1.11 LoteCompra(lote->Lote.numero, compra->Compra.id)

  Sem dependências funcionais não triviais. Não há dependências não triviais para a relação LoteCompra, pois ela apenas associa o lote à compra. Está em conformidade com o modelo relacional.

- 1.12 LoteLoja(lote->Lote.id, loja->Loja.id, stock)
    - lote : loja, stock

      A dependência é válida, pois o lote determina todos os outros atributos.

  Conclusão para LoteLoja: A dependência funcional está válida.

- 1.13 Venda(id, lote->Lote.id, funcionario->Funcionario.nif, cliente->Cliente.nif, quantidade, data, preco)
    - id : lote, loja, funcionario, cliente, quantidade, data
      A dependência é válida, pois o ID da venda determina todos os outros atributos.

  Conclusão para Venda: A dependência funcional está válida.

---

2. Verificação da BCNF ou 3NF

Agora, vamos verificar a normalização das relações.

- 2.1 Entidade

  Está em BCNF porque a dependência funcional tem uma chave candidata como determinante.

- 2.2 Cliente

  Está em BCNF porque não há dependências não triviais (apenas uma chave estrangeira).

- 2.3 Fornecedor

  Está em BCNF porque a dependência nif : nib tem o NIF como chave primária.

- 2.4 Ingrediente

  Não está em BCNF, pois a dependência nome : id, alergenio, calorias, stock viola a BCNF (se nome não for único). Porém, está em 3NF.

- 2.5 Compra

  Está em BCNF.

- 2.6 Produto

  Pode não estar em BCNF, dependendo da unicidade do nome. Caso o nome não seja único, não está em BCNF, mas estaria em 3NF.

- 2.7 ProdutoIngrediente

  Está em BCNF.

- 2.8 Loja

  Não está em BCNF, pois a dependência morada : id, contacto, numFuncionarios, totalSalario pode violar a BCNF (se morada não for única). Está em 3NF.

- 2.9 Funcionario

  Está em BCNF.

- 2.10 Lote

  Está em BCNF.

- 2.11 LoteCompra

  Está em BCNF.

- 2.12 LoteLoja

  Está em BCNF.

- 2.13 Venda

  Está em BCNF.

Conclusão Final:

- Ingrediente e Produto não estão em BCNF, mas estão em 3NF.
- Loja também não está em BCNF, mas está em 3NF.

Apreciação e modelo final

Na análise das dependências funcionais, a IA generativa notou que, para que nos pontos 1.4 e 1.6 as dependências funcionais fossem válidas, o atributo nome em cada uma das relações devia ser único. Sabe-se que a cada nome de um ingrediente apenas pode corresponder um ingrediente e a cada nome de um produto corresponde apenas um produto. Assim, ambos estes atributos são únicos e, por isso, as dependências funcionais destas relações são válidas.

Já no ponto 1.8, a situação é semelhante. Sabe-se que a cada morada deve apenas corresponder uma loja, ou seja, não há duas ou mais lojas na mesma morada. Por isso, morada é um atributo único, sendo a dependência funcional válida. Podemos assim verificar que todas as dependências funcionais propostas são válidas.

Como foram cometidos erros na análise das dependências funcionais pela IA generativa, foram providenciados novamente o esquema relacional e as correspondentes dependências funcionais, com as devidas clarificações, para que fosse possível uma nova análise.

Como seria de esperar, o modelo reconheceu que com as alterações nenhuma das dependências funcionais violava a BCNF. Foi, no entanto, ignorada a dependência funcional `contacto : id, morada, numFuncionarios, totalSalario.` No entanto, como contacto é uma superchave da relação, então esta dependência funcional também é válida. A conclusão de que a relação está em BCNF mantém-se, portanto, inalterada.

Como verificado pela IA generativa, no modelo relacional obtido após a decomposição, todas as relações se encontram em BCNF ou em 3NF. Assim sendo, concluímos que o modelo relacional apresentado anteriormente era bastante robusto, bastando excluir o atributo `loja` da classe `Venda`.

Criação da base de dados

Versão inicial

A versão inicial da base de dados encontra-se em anexo no ficheiro `create1.sql`.

Integração de IA generativa

Prompt:

Traduz este modelo relacional num ficheiro .sql que cria uma base de dados em SQLite3:

[Modelo obtido no passo anterior]

Considera também, sempre que aplicáveis em SQLite3, as seguintes restrições:

- NIF tem 9 dígitos
- NIB tem 21 dígitos
- Contacto tem 9 dígitos
- A data de uma venda deve ser maior do que a data a data de produção do lote correspondente
    - A quantidade de uma venda deve ser menor ou igual do que a quantidade do lote correspondente
    - O preço avulso deve ser maior ou igual ao preço em quantidade
    - A família de um produto é um de 'Padaria', 'Pastelaria' ou 'Confeitaria'

ChatGPT:

Aqui está um script SQL que cria a base de dados conforme o modelo relacional que descreveste, incluindo as restrições especificadas:

[Ficheiro em anexo: anexo1.sql]

Explicação das restrições

- NIF, NIB, e Contacto: Especificado o número de dígitos através de CHAR(n) e verificações CHECK.
- Data e Quantidade da Venda: As restrições CHECK garantem que a data da venda é maior ou igual à data de produção e que a quantidade não excede a do lote.
- Preço Avulso e Preço por Quantidade: A verificação CHECK assegura que o preço avulso é maior ou igual ao preço por quantidade.
- Família do Produto: Restrição em CHECK para garantir apenas valores permitidos ('Padaria', 'Pastelaria', 'Confeitaria').

Este script deve criar a base de dados no SQLite3 com todas as tabelas e restrições requeridas.

Prompt:

Porque usas CHAR(9) em vez de TEXT para campos com largura fixa, se já apresentas também uma restrição para o mesmo efeito logo a seguir?

ChatGPT:

Optar por CHAR(n) em vez de TEXT para campos de largura fixa, mesmo quando existe uma restrição CHECK para a contagem de dígitos, pode ter alguns motivos, mas, em SQLite, a vantagem prática é limitada devido à forma como o banco de dados lida com tipos de dados.

Aqui estão algumas considerações para a escolha de CHAR(n):

- Sinalização de Intenção: CHAR(n) indica claramente que o campo foi projetado para armazenar um valor de comprimento fixo. Isso pode ajudar outros desenvolvedores a entenderem a estrutura da base de dados e o tipo de dados esperado, aumentando a legibilidade do esquema.
- Compatibilidade: Em outros sistemas de bancos de dados (como MySQL ou PostgreSQL), CHAR(n) pode ter benefícios adicionais, como otimização de armazenamento e verificação automática de comprimento, pois esses sistemas tratam CHAR(n) de forma mais estrita do que o SQLite, que armazena valores de forma dinâmica.
- Validação Extra para o Usuário: Apesar de o CHECK garantir o comprimento, usar CHAR(n) mostra a intenção de comprimento fixo no nível do tipo de dado, funcionando como uma "dupla validação".

No SQLite, TEXT e CHAR(n) são tratados de maneira semelhante em termos de armazenamento. Assim, poderia simplificar o script usando TEXT e aplicando as restrições de comprimento com CHECK.

Melhoria

Satisfeitos com o resultado, implementámos a mudança de `text` para `char(n)` no programa de criação da base de dados e os checks transversais a várias tabelas (por exemplo `CHECK (quantidade <= (SELECT quantidade FROM Lote WHERE Lote.id = lote))`), que não sabíamos ser possíveis. Quando corremos o programa fomos surpreendidos pela seguinte mensagem de erro:

```
Parse error near line 135: subqueries prohibited in CHECK constraints
```

As sugestões de checks transversais eram, afinal, proibidas! Voltámos então a abordar o ChatGPT.

Prompt:

Ao correr o código sugerido recebi a mensagem de que "subqueries prohibited in CHECK constraints".

ChatGPT:

O SQLite não permite subconsultas (subqueries) dentro de restrições CHECK, o que impede a implementação direta das restrições de quantidade e data de venda no esquema que sugeri. Podemos resolver isso de duas maneiras principais:

- Mover a Lógica para Triggers: Podemos usar TRIGGERS para implementar essas restrições. Triggers são adequados para verificar condições complexas e podem ser usados para impedir inserções ou atualizações que não satisfaçam as condições.
- Implementação na Lógica de Aplicação: Outra abordagem seria implementar essas verificações no código da aplicação que interage com a base de dados. No entanto, isso pode não ser ideal se quisermos garantir a integridade apenas no nível da base de dados.

Versão final

Visto que estas restrições apenas podem ser implementadas usando triggers, que não são utilizados no trabalho, revertemos essas alterações. As restantes sugestões, como o uso de `CHAR(9)`, foram mantidas. A versão final pode ser consultada no ficheiro em anexo: `create2.sql`.

População da base de dados

A versão inicial de *data loading* encontra-se em anexo no ficheiro populate1.sql.

O objetivo deste primeiro esforço foi incluir *tuples* em todas as tabelas, tendo como ponto de partida a versão final da criação da base de dados. Os atributos que indicam preço têm o valor representado em euros e os que indicam quantidade estão expressos em unidades quando se referem a produtos e em Kg ou litros para ingredientes, dependendo de cada. Na relação `produto_ingrediente`, as quantidades inseridas correspondem à receita para uma unidade.

Integração de IA generativa

Prompt:

Cria um ficheiro .sql que popule a base de dados criada pelo ficheiro em anexo. Tem em conta que há atributos que dependem de outros da seguinte maneira:

- na classe ingrediente, o atributo stock deve ser inicializado a zero e ser incrementado de acordo com o atributo de quantidade dos tuples da classe compra que estejam relacionados cada ingrediente.
- na classe compra, o atributo preco deve ser o produto dos atributos quantidade e precoKg.
- na classe produto, o atributo calorias deve ser a soma dos produtos da quantidade de cada um dos seus ingredientes (como descritos na classe produto_ingrediente) com as suas calorias (como especificadas no atributo calorias da classe ingrediente).
- na classe loja, os atributos nfuncionarios e totalSalario são, respetivamente, a quantidade de funcionarios relacionados com a loja e a soma dos seus salarios.
- na classe funcionario, o atributo idade depende da dataNascimento.
- na classe lote, a dataValidade é obtida quando adicionamos a validade do produto do lote à dataProducao do lote.
- a classe lote_loja representa que lote está em que loja e, por isso, o stock de cada lote em cada loja deve ser parte (ou o todo) da quantidade de produto no lote em questão, diminuindo de cada vez que há uma venda desse produto nessa loja.
- na classe venda, o preco depende da quantidade vendida. Se, para o produto em questão, a quantidade de venda estiver abaixo do limiarTrocaPreco, o preco da venda é obtido através do produto do precoAvulso e da quantidade. Caso contrário, o preco da venda é o produto do precoQuantidade e da quantidade.

Para além disso, os valores de quantidades que se referem a produtos devem ser exprimidos em unidades. As quantidades de ingredientes são expressas em Kg ou litros, dependendo do ingrediente em questão. Na relação produto_ingrediente, a quantidade deve ser a quantidade (em Kg ou litros) de um dado ingrediente do produto presente numa única unidade do produto.

Assegura que cada tabela tem pelo menos um tuplo.

ChatGPT:

Vou analisar o ficheiro SQL que carregou e, em seguida, criar um ficheiro .sql para popular a base de dados de acordo com as regras fornecidas. Um momento enquanto verifico o conteúdo do ficheiro.

O arquivo contém instruções SQL para criar tabelas e estabelecer relações, incluindo a remoção de tabelas existentes. A estrutura está organizada com tabelas como entidade, cliente, fornecedor, ingrediente, entre outras.

Agora, com base no esquema e nas dependências mencionadas, vou criar um script para popular essas tabelas, garantindo que os atributos calculados sejam determinados conforme especificado. Um momento enquanto preparo o arquivo.

O script SQL para popular a base de dados foi gerado com sucesso. Você pode baixá-lo usando o link abaixo:

[ficheiro em anexo: anexo2.sql]

Se precisar de ajustes ou adicionar mais dados, é só avisar!

Prompt:

Corrige o ficheiro populate.sql que criaste para que os nomes dos atributos inseridos nas tabelas correspondam com o nome das colunas definidas no *script* SQL original.

Para além disso, certifica-te que:

- os *constraints* de chaves estrangeiras do ficheiro create2.sql são respeitadas, nomeadamente na classe funcionario, que tem um chave primária nif que é também chave estrangeira para a classe entidade.
- todos os constraints NOT NULL são respeitados, ou seja, que nenhum tuple inserido tenha algum atributo nulo. Quando o atributo tem de ser alterado posteriormente, inicializá-lo a zero se necessário.
- os valores para as calorias nos tuples da classe ingredientes correspondem às calorias para um Kg ou um litro do ingrediente, para que os valores obtidos para os produtos estejam corretos.
- são adicionados mais tuples às tabelas.

ChatGPT:

Vou ajustar o arquivo populate.sql com base nas novas especificações, garantindo:

1. Consistência nos nomes dos atributos com o esquema do arquivo create2.sql.
2. Respeito às restrições de chaves estrangeiras, incluindo o relacionamento entre funcionario e entidade.
3. Preenchimento de todos os atributos NOT NULL com valores válidos.
4. Ajuste das calorias dos ingredientes para corresponderem a valores por kg/litro.
5. Inclusão de mais tuples em todas as tabelas.

Um momento enquanto faço as alterações.

O arquivo corrigido foi gerado com sucesso. Você pode baixá-lo usando o link abaixo:

[ficheiro em anexo `anexo3.sql`]

Se precisar de mais ajustes ou modificações, estou à disposição!

Análise

Nos ficheiros gerados por estas *prompts* encontrámos os seguintes problemas:

- Os nomes dos atributos nas tabelas e nas *queries* estavam às vezes errados ou não existiam;
- Não foram respeitadas restrições de chaves estrangeiras nem as restrições NOT NULL. Por vezes, as chaves estrangeiras não existiam e, muitas vezes, atributos que dependiam de outros ficavam em branco por não terem sido calculados.

Para além disso, em geral, os valores inseridos nas tabelas (como a quantidade de ingredientes em cada produto e os valores calóricos) não correspondiam com a realidade, algo que tentámos assegurar na nossa tentativa inicial.

No entanto, os ficheiros gerados implementaram instruções do tipo UPDATE de maneira clara e mais simples do que as que tínhamos escrito, usando variáveis de tabela e a operação CASE, que foi especialmente útil para computar os preços associados a cada *tuple* da classe venda. Também expandimos a classe loja.

Por não termos encontrado grandes frutos destas *prompts*, continuamos o diálogo, desta vez fornecendo o nosso próprio ficheiro para a população da base de dados:

Prompt:

Para a população da base de dados criada pelo anexo anterior, create2.sql, foi escrito o código em anexo. Sugira alterações para melhorá-lo.

ChatGPT:

Aqui estão algumas sugestões para melhorar o script fornecido, garantindo sua funcionalidade, organização e aderência às melhores práticas:

Correções de Sintaxe e Funcionalidade

1. Erros em Atualizações (UPDATE ... FROM ...)

- O comando UPDATE ... SET ... FROM ... usado não é suportado em SQL padrão (incluindo SQLite). Em SQLite, as atualizações baseadas em outras tabelas precisam ser reescritas com subconsultas correlacionadas. Por exemplo:

```
UPDATE loja
    SET nfuncionarios = (SELECT COUNT(*) FROM funcionario WHERE funcionario.loja = loja.id);
```

Substitua todas as declarações UPDATE ... SET ... FROM ... dessa maneira.

2. Cálculo de Idade
   - O cálculo da idade no INSERT INTO funcionario com strftime e timediff está incorreto para SQLite. Substitua-o por uma abordagem funcional, como:

```
CAST((strftime('%Y', 'now') - strftime('%Y', dataNascimento)) AS INTEGER)
```

Para lidar corretamente com aniversários futuros, ajuste com mais lógica.

3. Datas no Formato ISO 8601
   - Ao usar strftime('%F'), garanta que todas as datas estejam no formato ISO 8601. Verifique todas as datas fornecidas para consistência.

4. Manipulação de Estoques
   - Atualize estoques de ingredientes e lotes com lógica robusta:
     - Evite valores negativos ao subtrair estoques, validando antes da atualização.
     - Implemente restrições ou validações para manter consistência entre tabelas.

Melhorias na Organização

1. Separação de Lógicas
   - Divida o script em seções bem definidas:
     - População inicial de dados (e.g., loja, entidade, ingrediente).
     - Atualizações calculadas (e.g., estoque, calorias, preços).
     - Registros transacionais (e.g., compras, vendas).
   - Adicione comentários descritivos antes de cada seção.

2. Reuso de Consultas
   - Considere usar WITH (Common Table Expressions) para consultas repetitivas, como calcular preços de venda:

```
WITH Precos AS (
    SELECT venda.id AS venda_id,
           CASE
           WHEN venda.quantidade < produto.limiarTrocaPreco THEN produto.precoAvulso
               ELSE produto.precoQuantidade
           END AS preco
    FROM venda
    JOIN lote ON venda.lote = lote.id
    JOIN produto ON lote.produto = produto.id
)
UPDATE venda
```

SET preco = (SELECT preco FROM Precos WHERE venda.id = Precos.venda_id);

3. Manutenção de Legibilidade
   - Use indentação consistente para melhorar a leitura.
   - Limite o número de operações em um único comando SQL.

Verificações de Conformidade
1. Restrições de Integridade
   - Certifique-se de que:
     - Todas as inserções respeitam as restrições NOT NULL e FOREIGN KEY.
     - Dados relacionados são consistentes (e.g., IDs de lotes, produtos e funcionários correspondem).

2. Validações de Dados
   - Adicione verificações para evitar erros, como estoques negativos ou dados duplicados. Exemplo:

```
INSERT INTO ingrediente (nome, alergenio, calorias)
SELECT 'Farinha de Trigo', 1, 3640
 WHERE NOT EXISTS (SELECT 1 FROM ingrediente WHERE nome = 'Farinha de Trigo');
```

3. Valores de Quantidades
   - Garanta que quantidades estejam no formato correto:
     - Ingredientes em kg/litro.
     - Produtos em unidades.

Outras Sugestões
1. Melhoria no Cálculo de Calorias
   - Verifique se o cálculo de calorias em produtos está correto e otimizado:

```
UPDATE produto
SET calorias = (
    SELECT SUM(ingrediente.calorias * produto_ingrediente.quantidade)
     FROM produto_ingrediente
    JOIN ingrediente ON produto_ingrediente.ingrediente = ingrediente.id
    WHERE produto_ingrediente.produto = produto.id
);
```

2. Mensagens de Log
   - Adicione logs para verificar o progresso da execução:

```
SELECT 'Lotes e vendas atualizados com sucesso!' AS LogMessage;
```

3. Testes com Conjuntos de Dados Maiores
   - Insira mais registros em tabelas (e.g., novos produtos, compras, e clientes) para testar o desempenho e consistência.

```
Se precisar de exemplos específicos ou ajustes detalhados para alguma parte do código, posso ajudá-lo!
```

Análise

Desta lista de melhorias, considerámos adequado implementar:

- a operação CAST ao calcular a idade dos funcionários;
- adicionar comentários antes das operações;
- regularizar a indentação e os statements de UPDATE.

Finalmente, elaboramos o ficheiro para *data loading* definitivo, em anexo como `populate2.sql`.

Conclusão

Neste projeto criámos uma base de dados para uma padaria. A estrutura do projeto forçou-nos a uma construção gradual que segue boas práticas. É um método de trabalho que poderemos vir a usar no futuro, quando necessitarmos de construir outras bases de dados.

Um dos pontos principais do projeto era a utilização de IA generativa. Esta revelou-se, na verdade, muito pouco útil. Os resultados da aplicação desta tecnologia eram na sua maioria redundantes, sendo mesmo por vezes simplesmente erróneos, tendo sido sugeridas abordagens que continham erros sintáticos de SQL. No entanto, alguns dos resultados foram úteis. Utilizada como uma ferramenta para verificar a correção dos procedimentos realizados, esta tecnologia pode aumentar a confiança nos resultados obtidos, como, por exemplo, ao verificar se uma relação se encontra em BCNF ou 3NF. Destacamos também a insistência em seguir boas práticas na escrita de *scripts* SQL e a descoberta de tecnologias que ainda não tínhamos abordado nas aulas, como os *triggers*. Ainda assim, concluímos que o tempo necessário para obter resultados úteis era maior do que o tempo poupado pela utilização da tecnologia.

Não pretendemos, com isto, afirmar que a utilização de IA generativa no nosso trabalho tenha sido inútil. Pelo contrário, foi um exercício importante para explorar os pontos fortes e fracos destas ferramentas, permitindo-nos compreender melhor o seu funcionamento e aprender a tirar pleno partido do seu potencial estando cientes das suas limitações.